

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

Звіт
З самостійної роботи №1
з дисципліни: «Основи програмної інженерії»
на тему:
«Розгортання застосунку на сервері або у хмарі»

Виконала: ст. гр. ПЗПІ-25-3
Гиренко В. О.

Прийняв: Гребенюк В. О.

Харків 2026

РОЗГОРТАННЯ ЗАСТОСУНКУ НА СЕРВЕРІ АБО У ХМАРІ

МЕТА РОБОТИ

Набуття практичних навичок із розгортання веб-застосунку на хмарній PaaS-платформі. Оволодіння процесом перетворення програмного модуля на REST API та конфігурування середовища розгортання. Отримання досвіду верифікації працездатності розгорнутого застосунку через публічний URL.

ХІД ВИКОНАННЯ РОБОТИ

1.1 URL працюючого застосунку

У результаті виконання кроків розгортання веб-обгортки (REST API) на базі модуля з ЛР 02–04 та після успішної реєстрації на хмарній платформі Render, було отримано глобальну URL-адреса сервісу:

<https://hookcraft-api-viel.onrender.com>

1.2 Скріншот відповіді /health endpoint у браузері

Для реалізації веб-обгортки у файлі Program.cs було налаштовано базовий кореневий ендпоінт (GET /), який виконує роль Health Check. Перевірка працездатності розгорнутого хмарного застосунку була виконана безпосередньо через веб-браузер шляхом переходу за публічним URL.



HookCraft REST API успішно запущено!



Рисунок 1.1 – Верифікація працездатності хмарного сервера через веб-браузер

1.3 Результат успішного виконання HTTP-запиту до основного API-endpoint хмарного сервера

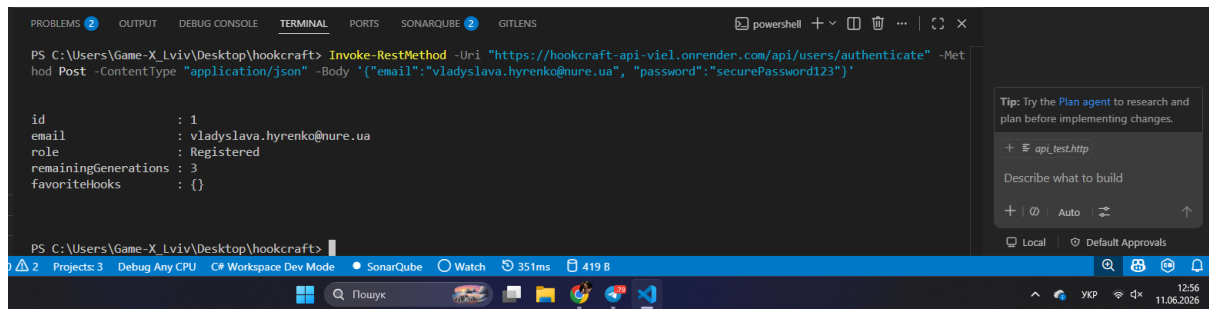


Рисунок 1.2 – Скріншот успішної відповіді основного API-endpoint

1.4 Dashboard хмарної платформи з індикатором «Deploy successful»

Після створення конфігураційних файлів та запуску процесу релізу, хмарна платформа Render виконала автоматичне зчитування інструкцій, компіляцію C# коду та розгортання мікросервісу.

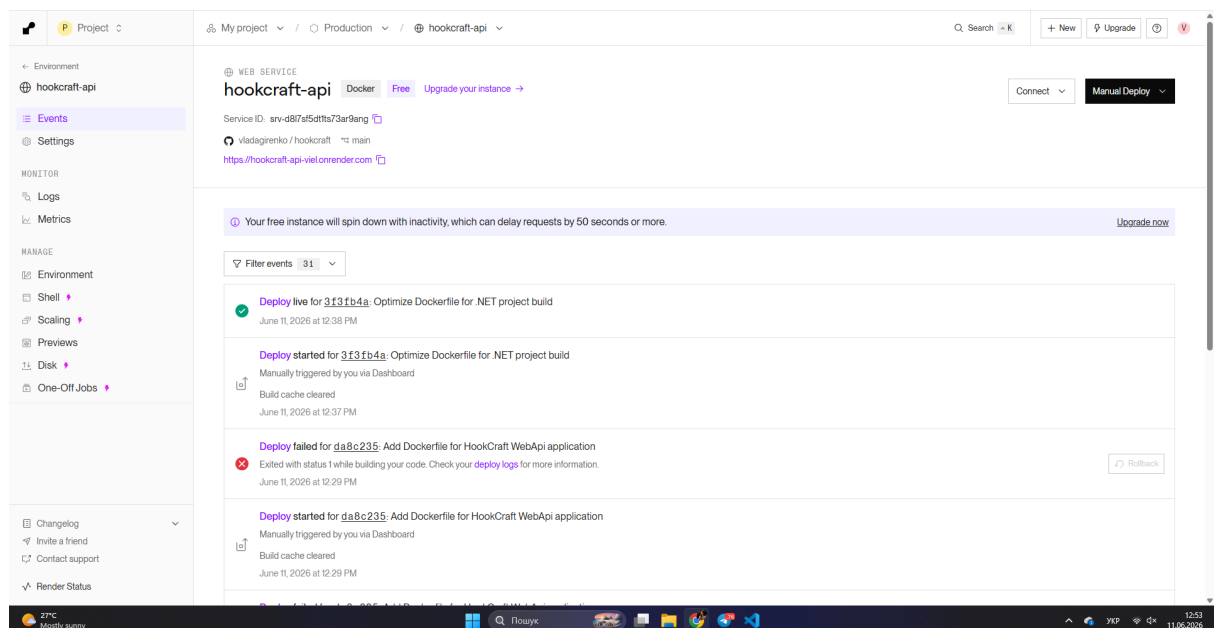


Рисунок 1.3 – Render Dashboard із індикатором успішного розгортання «Live»

1.5 Конфігураційні файли проекту

Оскільки архітектура проекту розроблена на базі екосистеми .NET 8 (C#), менеджмент пакетів, версії платформи та зв'язки між модулем бізнес-логіки та веб-сервером реалізовано через XML-структури файлів проєктів *.csproj (зокрема HookCraft.WebApi.csproj).

Проте, для виконання бонусного завдання (п. 5.2.6), у кореневому каталозі репозиторію було створено конфігураційний файл Dockerfile.

hookcraft / Dockerfile 

 vladagirenko Optimize Dockerfile for .NET project build

Code Blame 18 lines (11 loc) • 406 Bytes

```
1
2 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build-env
3 WORKDIR /app
4
5 COPY . ./
6
7 RUN dotnet restore src/HookCraft.WebApi/HookCraft.WebApi.csproj
8 RUN dotnet publish src/HookCraft.WebApi/HookCraft.WebApi.csproj -c Release -o out
9
10
11 FROM mcr.microsoft.com/dotnet/aspnet:8.0
12 WORKDIR /app
13 COPY --from=build-env /app/out .
14
15 ENV ASPNETCORE_URLS=http://+:8080
16 EXPOSE 8080
17
18 ENTRYPOINT ["dotnet", "HookCraft.WebApi.dll"]
```

Рисунок 1.4 – Код створеного Dockerfile

1.6 Посилання на Git-репозиторій з кодом

<https://github.com/vladagirenko/hookcraft>

1.7 Файл .github/workflows/deploy.yml (CI/CD pipeline) (п. 5.2.7)

Для автоматизації розгортання при кожному push до головної гілки main було налаштовано автоматичний пайплайн конвеєра GitHub Actions.

 vladagirenko Create deploy.yml 

Code

Blame

32 lines (24 loc) · 679 Bytes

```
1   name: .NET Core CI/CD Pipeline
2
3   on:
4     push:
5       branches: [ "main" ]
6
7   jobs:
8     build-and-test:
9       runs-on: ubuntu-latest
10
11      steps:
12        - name: Checkout repository code
13          uses: actions/checkout@v4
14
15        - name: Setup .NET Core SDK
16          uses: actions/setup-dotnet@v4
17          with:
18            dotnet-version: '8.0.x'
19
20        - name: Restore dependencies
21          run: dotnet restore
22
23        - name: Build projects
24          run: dotnet build --no-restore --configuration Release
25
26        - name: Run Unit Tests
27          run: dotnet test --no-build --verbosity normal
28
29        - name: Trigger Render Deploy Hook
30          if: success()
31          run: |
32            curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

Рисунок 1.5 – Код налаштованого файлу (.github/workflows/deploy.yml)

ВИСНОВКИ

Під час виконання цієї самостійної роботи я здобула практичний досвід створення повноцінних хмарних вебсервісів на базі платформи .NET 8, навчившись обгортати готову бізнес-логіку програми в сучасні REST API ендпоінти. Я детально розібралася з принципами контейнеризації додатків, самостійно розробивши багатоетапний Dockerfile для ізоляції та оптимізації роботи мікросервісу. На практиці освоїла хмарне розгортання проєктів на платформі Render та навчилася тестувати працездатність віддаленого API за допомогою консольних утиліт через PowerShell.

Крім того, я опанувала інструменти автоматизації (CI/CD), побудувавши автоматичний конвеєр GitHub Actions, який самостійно запускає Unit-тести перед кожним оновленням програми у хмарі. Ця робота допомогла мені зрозуміти повний життєвий цикл розробки програмного забезпечення від написання коду до його автоматичного релізу в інтернеті.